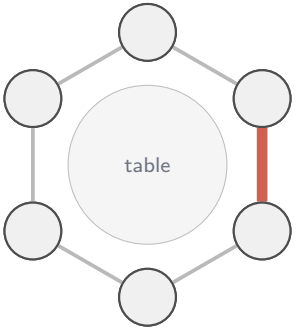
Neighborhood Stability in Assignments on Graphs

Haris Aziz, Grzegorz Lisowski, Mashbat Suzuki, Jeremy Vollen
arXiv:2407.05240, 2024

Presented by Ashik Stenny (230224) and Achyuth H Warrier (230052)

CS698I: Relational Structures in Games · IIT Kanpur · Instructor: Sunil Simon
11 November 2026

The dinner-party problem



“swap?”

- A host seats n guests on n fixed seats.
- Each guest **likes** some of the others: a yes/no list.
- Happiness = how many liked guests sit **next to you**.
- Two neighbours who would *both* be happier after swapping seats will swap.

Goal: a seating where no two neighbours want to swap.

Why study this?

Where the problem appears

- **Roles in an organisation:** members don't mind which project they get, only which colleagues they work with.
- **Seating arrangements** (Bodlaender et al. 2020) and several follow-up papers.
- A relative of **hedonic games**: agents care only about *who* they are grouped with.

Why a new notion of stability

- Full stability often fails: Berriaud et al. found many non-existence results on paths and cycles.
- Precedent: in Schelling games, allowing only **local swaps** makes equilibria exist on more graphs (Bilò et al.).
- A swap needs the two agents to interact, so only neighbours can arrange one.

Binary preferences = each agent approves a set of others, as studied in committee voting, matching and item allocation.

The question the paper asks

"For which classes of seat graphs is a neighborhood stable assignment guaranteed to exist under binary preferences?"

Setting the scene

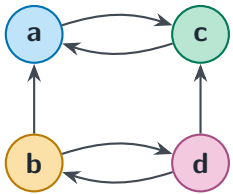
- The model, on a 4-guest example
- Stability can fail
- Letting guests swap can loop forever

Positive results

- Round tables (cycles): always solvable
- Long benches (paths)
- A general condition; discussion

Two graphs: who likes whom, and who sits where

Preference graph $\mathcal{P}$ (directed)



$i \to j$: guest i likes guest j

Seat graph G (undirected)

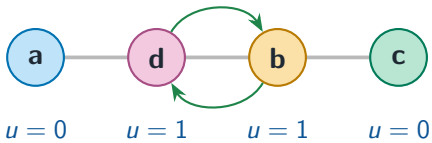


edge = the two seats are next to each other

- An **assignment** π puts each of the n agents on one of the n seats (a bijection).
- **Running example** (paper's Fig. 2): $a \leftrightarrow c$, $b \leftrightarrow d$, $b \to a$, $d \to c$; seats in a row.

Utility = number of liked neighbours

$$u_i(\pi) \;=\; \big|\{\, j \in N_\pi(i) : i \to j \,\}\big| \qquad N_\pi(i) = \text{agents seated next to } i$$



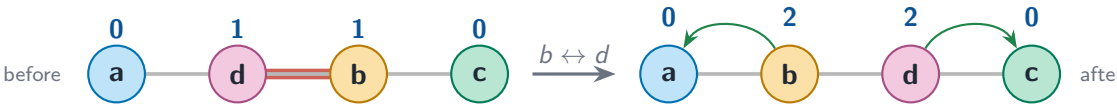
green = a liked neighbour

- a likes only c, who is far away: $u_a = 0$. d likes b next to it: $u_d = 1$.

An additively separable hedonic game with 0/1 values, where your "group" is your seat neighbourhood.

Swaps, envy and blocking pairs

- $\pi^{i \leftrightarrow j}$: π with i and j exchanging seats.
- i **envies** j if $u_i(\pi^{i \leftrightarrow j}) > u_i(\pi)$.
- i, j form a **blocking pair** if each envies the other.



- d trades neighbour a (not liked) for c (liked): $1 \to 2$.
- b trades neighbour c (not liked) for a (liked): $1 \to 2$.
- Both gain, so b, d is a blocking pair.

Neighbourhood stability

Definition (neighbourhood stable, NS)

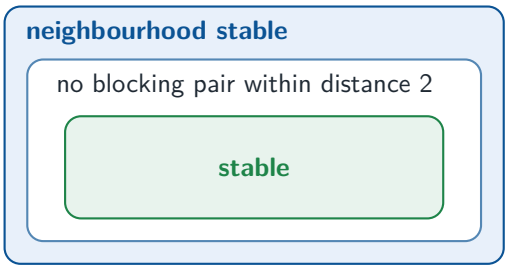
An assignment π is **neighbourhood stable** if no blocking pair sits on **adjacent** seats.

Compare: π is **stable** if there is no blocking pair at all.

Why only neighbours?

- To swap, two guests must first talk and agree.
- At a table you only talk to the people beside you.
- Guests may not even know where everyone else is sitting.

How the notions relate



- Running example after the swap, a, b, d, c: **stable**, hence also NS.
- NS is the weakest of the three, so it is the most likely to exist.
- Even checking whether a *stable* seating exists is computationally hard.

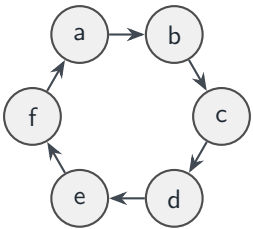
Why binary (yes/no) preferences?

Richer preferences already break NS on tiny instances:

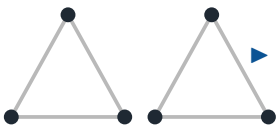
- **Three non-negative values**: 4 agents on a cycle with no NS assignment (from Berriaud et al.'s proof).
- **Values** ± 1 (dislike allowed): fails already on a path of three seats.
- Two non-negative values are the same as binary, without loss of generality.

So binary preferences are where positive results might live.

NS is genuinely weaker: two tables of three



preferences: a directed 6-cycle



seats: two triangles

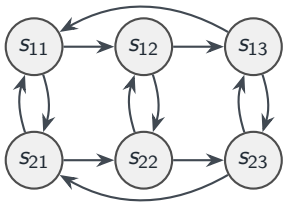
- **No stable assignment:** some two agents at different tables both have utility 0, and they block.
- **Every assignment is NS:** neighbours share a table, and swapping them changes nobody's neighbours.

Paper's Example 2, taken from Example 2 of Massand & Simon, *Graphical one-sided markets*, IJCAI 2019.

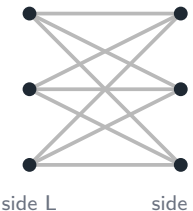
Theorem 1: NS assignments need not exist

Theorem 1

For every $n = 2t$ with $t \geq 3$ odd, there is an instance with n agents and no NS assignment.



each agent likes its **twin** and its **row successor**



seat graph $K_{3,3}$ (complete bipartite)

Paper's Fig. 1 ($t = 3$). We also checked all $6! = 720$ seatings: none is NS.

Theorem 1, key idea: only your side matters

- In $K_{t,t}$ your neighbours are *everyone on the other side*. So a seating is just a split into sides L and R.
- Everyone likes exactly 2 agents; utility = how many of them are across.

Two ways for i to envy an agent j across:

(C1) i likes **both** its favourites on its own side ($u_i = 0$). Then i envies *everyone* across: $0 \to 2$.

(C2) i likes **one** agent on its own side and also likes j. Then i envies j: $1 \to 2$.

Theorem 1, proof sketch: a blocking pair always appears

Case 1: some twins sit together

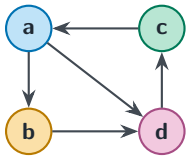
- t odd $\Rightarrow$ some other twin pair is split.
- Some agent then has $u = 0$ and envies the whole other side (C1).
- Following the twins around the row finds an agent that envies it back (C2).

Case 2: all twins split

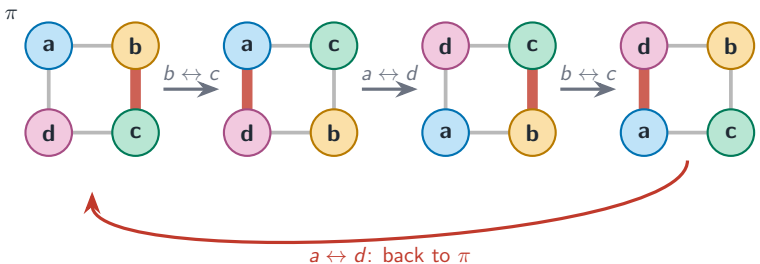
- t odd $\Rightarrow$ two *consecutive* agents of one row share a side.
- One of them and its twin envy each other (C2).

Both cases use that t is odd.

Proposition 1: letting guests swap can loop forever



preferences (Fig. 6)



- Red = the **only** blocking pair in that seating; both swappers gain.
- Swapping blocking neighbours can **cycle forever**, even on 4 seats.

What Proposition 1 tells us

- An **improvement cycle**: no potential can rise with every swap.
- From this start, *every* sequence of swaps loops.
- Yet NS seatings do exist here.

No finite improvement property (FIP), not even weakly acyclic.
So on cycles we need an **algorithm**, not dynamics.

Map of results

Seat graph	Guarantee	Algorithm
$K_{t,t}$, $t \geq 3$ odd	NS may not exist (Thm 1)	–
Cycle	NS always exists (Thm 2)	$O(n^3)$
Path	no blocking pair within distance 2 (Thm 3)	$O(n^2)$
Enough leaves	NS exists (Thm 4)	poly. if $\mathcal{P}$ is a DAG

DFVS = directed feedback vertex set (explained with Theorem 4); DAG = directed acyclic graph.
$O(n^2)$, $O(n^3)$: running time grows at most like the square / cube of the number of agents.

Theorem 2: round tables always work

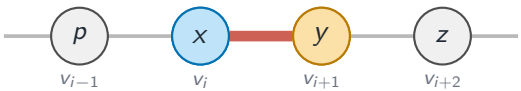
Theorem 2 (central result)

If the seat graph is a cycle, an NS assignment always exists and can be computed in $O(n^3)$ time.

Why this is surprising:

- Forbidding blocking pairs within distance 2 can fail on a 5-agent cycle (Berriaud et al.).
- With 3-valued preferences it fails on a 4-agent cycle.
- Swap dynamics can loop (Proposition 1).

The one tool we need: a swap trades one neighbour



When neighbours x, y swap on a cycle, they stay adjacent. So:

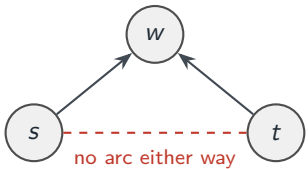
- x gives up p and gets z; y gives up z and gets p.

$$x, y \text{ block} \iff x \to z, \quad x \not\to p \quad \text{and} \quad y \to p, \quad y \not\to z$$

- **Obs. 2:** if x already likes p, then x will not envy y.
- **Obs. 1:** if $y \not\to p$ or $x \not\to z$, then x, y cannot block.

Two cases, depending on the preference graph

Case 1: there is a "V"

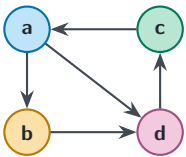


Two agents like the same w but neither likes the other.

Running example: $b \to a \leftarrow c$, and b, c unrelated.

Case 2: no "V"

Whenever two agents like the same person, one of them likes the other.



The looping instance (Fig. 6) is in Case 2: only d has two admirers, a and b, and $a \to b$.

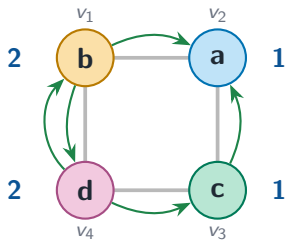
Testing every triple of agents decides the case in $O(n^3)$ time.

Case 1: a greedy seating

Algorithm (Case 1)

1. Seat s, w, t on v_1, v_2, v_3.
2. For $v_4, v_5, \ldots, v_n$: seat an unseated agent who **likes the previous seat's agent**; if there is none, seat anyone.

Run on the running example:
$s = b$, $w = a$, $t = c$. Who likes c? Only d, so d takes v_4.



Everyone gets their maximum possible utility.

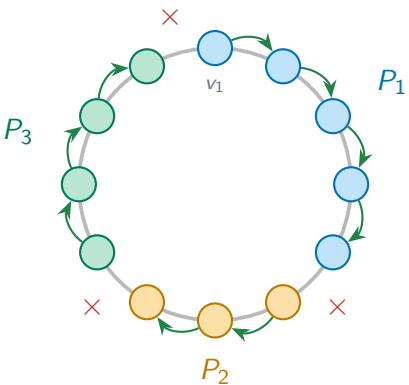
Why the greedy seating is NS

Recall: x, y (with p left of x) block only if $y \to p$ and $x \not\to p$.

- **Around the V:** $s \not\to t$, so s won't envy w; $t \not\to s$, so t won't envy w. s and t both like w, so neither envies its other neighbour.
- **Everywhere else:** take neighbours x (at v_i) and y (at v_{i+1}).
  - If $x \to p$: no envy from x (Obs. 2).
  - If $x \not\to p$: when x was chosen, *nobody* unseated liked p, including y. So $y \not\to p$ (Obs. 1).

Invariant: for every pair x, y, either x likes the agent on its left, or y does not. That is all NS needs.

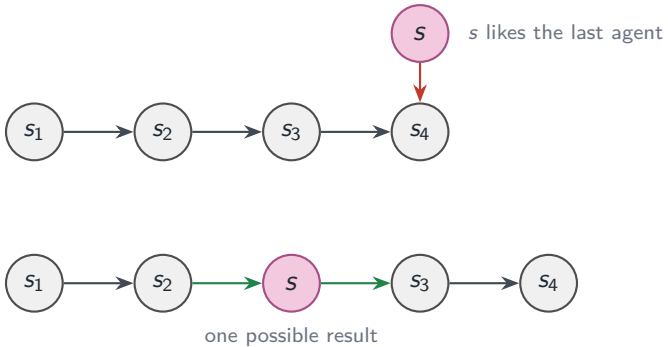
Case 2: wrap path partitions around the table



- **Path partition:** split the agents into directed paths (each agent likes the next one).
- $\Phi(P_1, \ldots, P_k)$**:** lay the paths around the cycle, one after another.
- **Minimal:** no path's tail likes another path's head (otherwise glue them).
- **Start:** repeatedly remove a maximal path. This gives a minimal partition.

Progress: R = number of agents who like their right neighbour.

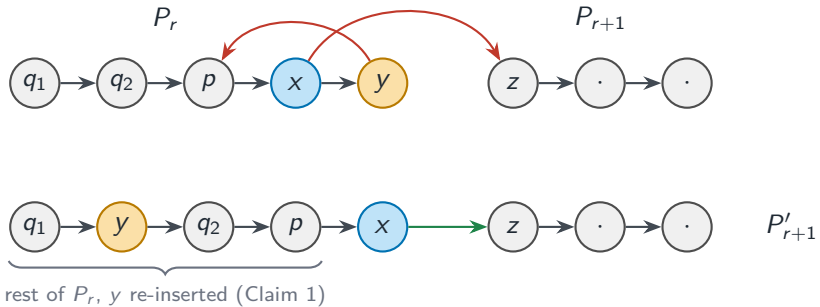
Claim 1: in Case 2 you can always squeeze in



Claim 1. If s likes the last agent of a path, s can be inserted into the path.

Why: s and s_3 both like s_4, so one likes the other (no V). Insert, or step back and repeat.

Lemma 1: repairing a blocking pair merges two paths



Case: x, y **in the same path.**

- Blocking forces y = tail, z = next head, and $y \to p$, $x \to z$ (red).
- Re-insert y (Claim 1), then glue $p \to x \to z$: **one fewer path**.

Finishing Theorem 2

- A blocking pair cannot straddle two paths: that would contradict minimality.
- $R \leq n$ and each repair raises R: at most n repairs, $O(n^3)$ total.
- Fig. 6: the path $c \to a \to b \to d$, wrapped around, is NS at once.

A progress measure, as for Hopfield networks, but with only $n+1$ values: no exponential search as in PLS (Polynomial Local Search).

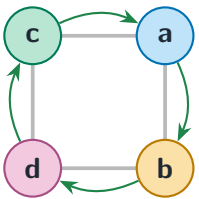


Fig. 6 instance, path $c \to a \to b \to d$ (swap dynamics looped here)

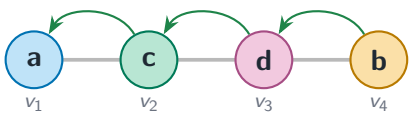
Theorem 3: benches (paths)

Theorem 3

If the seat graph is a path, an assignment with **no blocking pair within distance 2** exists and is computed in $O(n^2)$ time.

Same greedy: each next seat gets someone who likes the previous agent.

Running example: a, c, d, b.



fully stable (checked by brute force)

Swapping also converges on paths (Berriaud et al.), but maybe exponentially slowly. This paper settles their open question with a direct, fast construction.

Theorem 4: a condition for any seat graph

Theorem 4 (as stated)

If the number of **leaves** of G exceeds the DFVS number γ of $\mathcal{P}$, an NS assignment exists. If $\gamma = 0$ ($\mathcal{P}$ is a DAG), a *stable* assignment is computable in polynomial time.

DFVS (directed feedback vertex set): agents X whose removal leaves $\mathcal{P}$ acyclic; $\gamma =$ smallest $|X|$. **Leaf:** a seat with one neighbour.

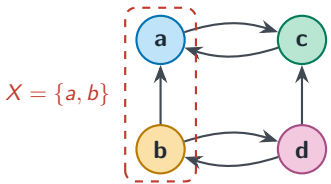
Algorithm

1. Seat X on leaves.
2. The rest form a DAG. Repeatedly a **sink** (likes nobody still unseated) picks its best free seat.

Why NS

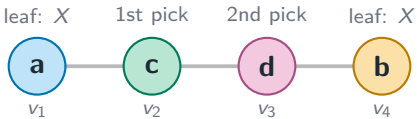
- A sink's friends are all seated before it picks, so it never envies a later agent.
- Swapping into a leaf leaves you next to only the agent you swapped with, so a leaf agent is never envied.

Theorem 4 on the running example, and a consequence



$a \leftrightarrow c$ and $b \leftrightarrow d$ are disjoint cycles, so

$\gamma = 2$



c (sink) sits by a; then d. Result a, c, d, b is NS.

- **Planar** preferences: $\gamma \leq \frac{3}{5}n$, so any seat graph with $\geq 60\%$ leaves works.

Same trick as colouring games on DAGs: sinks choose first.

Open questions (from the paper)

- **Other seat graphs:** grids, trees, and more conditions in the style of Theorem 4.
- **Efficiency:** how much welfare is lost by insisting on NS? Can we maximise welfare among NS seatings?
- **Beyond yes/no preferences:** on cycles NS can fail with cardinal values; on paths this is still open.

Summary

- **Neighbourhood stability:** only neighbours can swap. Weaker than stability, and natural.
- **Not always possible:** $K_{t,t}$ with t odd (Theorem 1). Swaps can loop even on 4 seats (Proposition 1).
- **Cycles:** always possible, $O(n^3)$, via path partitions and a progress measure (Theorem 2).
- **Paths:** even distance-2 stability, by a greedy algorithm (Theorem 3).
- **General:** NS exists if the seat graph has enough leaves compared with the DFVS number (Theorem 4).

Thank you! Questions?